

Estruturas de Dados em Python

Aprender a guardar, organizar e aceder à informação de forma eficiente — o ponto de partida para escrever programas verdadeiramente úteis.

Técnico de Desenvolvimento de Software

UC 00618 – Criar aplicações em linguagem de programação Python

Formador: Ricardo Morgado

Cofinanciado por:



O que vamos aprender hoje?

Esta sessão foi desenhada a pensar em ti — sem experiência prévia necessária. Vamos avançar passo a passo, sempre com exemplos do dia a dia.

01

Parte 1 — Fundamentos

O que são estruturas de dados e por que são tão importantes

03

Parte 3 — As 4 Estruturas

List, Tuple, Set e Dict — com exemplos práticos e exercícios

02

Parte 2 — Tipos Básicos

Revisão rápida de `str`, `int`, `float` e `bool`

04

Parte 4 — Prática Integradora

Exercícios completos para consolidar tudo o que aprendeste

Motivação: Porquê organizar informação? 💡

O problema

Imagina que precisas de guardar os nomes de 100 alunos. Criavas 100 variáveis diferentes? `aluno1`, `aluno2`... `aluno100`? Isso seria caótico e impossível de gerir!

A solução

As estruturas de dados permitem-nos guardar muita informação de forma organizada, numa única variável. É como ter uma pasta bem organizada em vez de papéis espalhados por todo o lado.

💡 Pergunta à turma: Como é que organizas os contactos no teu telemóvel? Já estás a usar uma estrutura de dados sem saberes!

O que são Estruturas de Dados?

Definição simples: São formas de guardar e organizar dados em memória para que possamos trabalhar com eles de forma eficiente.

List

Lista de compras — ordenada, pode mudar

Dict

Ficha de aluno — chave e valor

Set

Turma única — sem repetições

Tuple

Cartão de cidadão — não muda



Revisão Rápida: Tipos Básicos ⚡

Antes de avançarmos, vamos recordar os tipos de dados simples que já conheces. Cada variável em Python tem um tipo — e podemos descobri-lo com a função `type()`.

str

Texto

"Olá, mundo! "

int

Número inteiro

42

float

Número decimal

3.14

bool

Verdadeiro/Falso

True / False

A Função `type()`

A função `type()` diz-nos que tipo de dado está guardado numa variável. É como perguntar ao Python: "O que é isto?"

Exemplos de código

```
nome = "Ana"  
idade = 20  
altura = 1.65  
ativo = True
```

```
print(type(nome))    # <class 'str'>  
print(type(idade))   # <class 'int'>  
print(type(altura))  # <class 'float'>  
print(type(ativo))   # <class 'bool'>
```

Experimenta!

Abre o teu editor de Python e cria 3 variáveis diferentes:

1. O teu nome (str)
2. O teu ano de nascimento (int)
3. A tua altura em metros (float)

Usa `type()` para confirmar o tipo de cada uma.

Listas em Python

A lista é a estrutura de dados mais utilizada em Python. Pensa nela como uma lista de compras: tem uma ordem, podes adicionar ou remover itens, e podes ter o mesmo produto mais do que uma vez.

- 
1. MILK - whole fat
 2. YOGURT - greek
 3. AVOCADO - Hass
 4. ORANGES - navel
 5. STRAWBERRIES

Cofinanciado por:

O que é uma Lista?

Uma lista é uma coleção ordenada e mutável de elementos. "Ordenada" significa que os elementos têm uma posição fixa. "Mutável" significa que podemos alterá-la depois de criada.

Características

- Ordenada — os elementos têm índices
- Mutável — pode ser alterada
- Permite duplicados
- Pode guardar tipos diferentes

Como criar uma lista

```
# Lista de frutas
lista = ["maçã", "banana", "laranja"]

# Lista de números
numeros = [10, 20, 30, 40]

# Lista mista
mista = ["Ana", 20, True, 3.14]

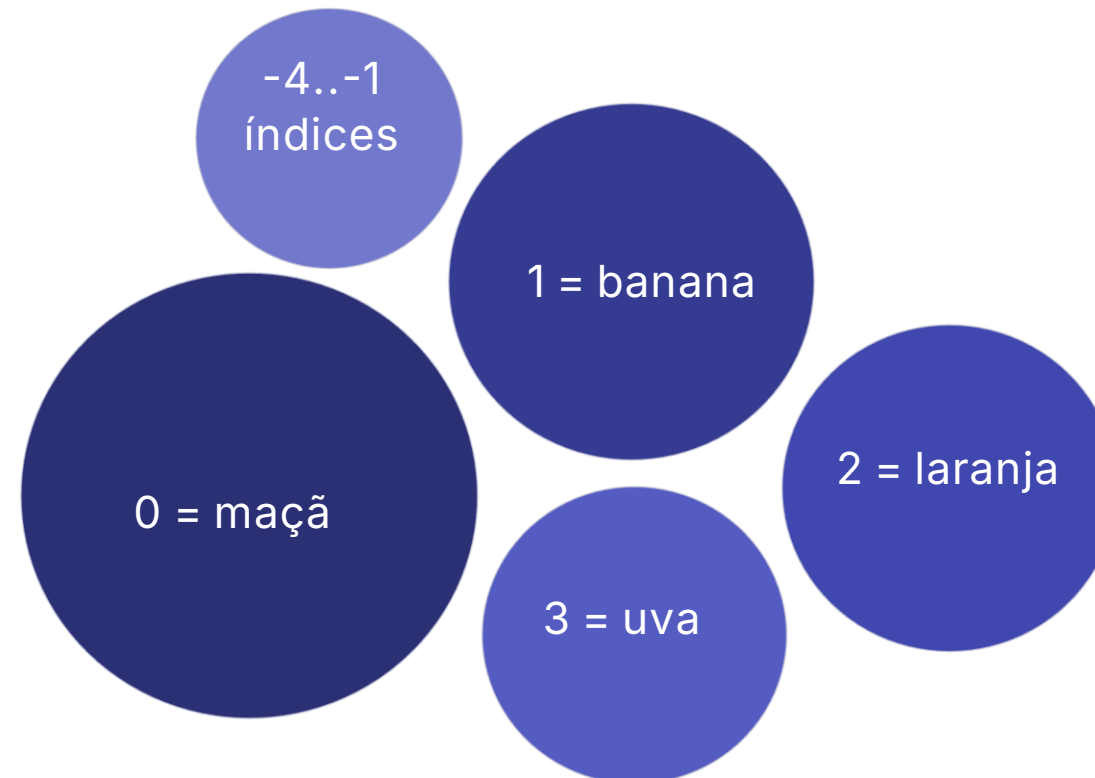
print(lista)
print(type(lista)) # <class 'list'>
```


Acesso por Índice

Cada elemento de uma lista tem um número de posição chamado índice.

Em Python, o primeiro elemento está na posição 0 — não 1! É um erro muito comum nos iniciantes.

```
frutas = ["maçã", "banana", "laranja", "uva"]  
  
print(frutas[0])    # maçã    (primeiro elemento)  
print(frutas[1])    # banana  (segundo elemento)  
print(frutas[-1])   # uva     (último elemento)  
print(frutas[-2])   # laranja (penúltimo elemento)
```



Os índices negativos contam a partir do fim da lista — muito útil quando não sabemos o tamanho da lista!

Cofinanciado por:

Slicing — Fatiar uma Lista

O slicing permite-nos extrair uma parte de uma lista. A sintaxe é `lista[início:fim]` — o elemento na posição `fim` não é incluído.

```
numeros = [10, 20, 30, 40, 50]

# Primeiros 3 elementos
print(numeros[0:3]) # [10, 20, 30]

# A partir do índice 2
print(numeros[2:])  # [30, 40, 50]

# Até ao índice 3
print(numeros[:3])  # [10, 20, 30]

# Toda a lista
print(numeros[:])   # [10, 20, 30, 40, 50]
```

Analogia

Imaginem uma fila de pessoas. O slicing é como dizer: "quero as pessoas da posição 2 até à posição 4". A pessoa na posição 4 fica de fora — só entram as que estão antes dela.

Métodos Essenciais das Listas

As listas têm métodos incorporados — funções especiais que nos permitem manipulá-las facilmente. São como os botões de um comando: cada um faz uma coisa específica.

append()

Adiciona um elemento no fim da lista

```
lista.append("manga")
```

remove()

Remove a primeira ocorrência de um valor

```
lista.remove("banana")
```

sort()

Ordena a lista por ordem crescente

```
lista.sort()
```

len()

Devolve o número de elementos

```
len(lista)
```

Exemplo Completo — Lista de Compras

```
# Criar lista de compras
compras = ["maçã", "banana", "leite"]
print("Inicial:", compras)

# Adicionar produto
compras.append("pão")
print("Após append:", compras)

# Remover produto
compras.remove("banana")
print("Após remove:", compras)

# Ordenar lista
compras.sort()
print("Ordenada:", compras)

# Quantos produtos?
print("Total:", len(compras), "produtos")
```

Resultado esperado

```
Inicial: ['maçã', 'banana', 'leite']
Após append: ['maçã', 'banana', 'leite', 'pão']
Após remove: ['maçã', 'leite', 'pão']
Ordenada: ['leite', 'maçã', 'pão']
Total: 3 produtos
```




Exercício Prático — Lista de Produtos

Chegou a altura de praticares! Segue as instruções passo a passo no teu editor.

1

Criar a lista

Cria uma lista chamada `produtos` com pelo menos 4 itens à tua escolha (por exemplo, artigos de papelaria)

2

Adicionar

Adiciona 2 novos produtos à lista usando `append()`

3

Remover

Remove um produto usando `remove()`

4

Ordenar e contar

Ordena a lista com `sort()` e imprime o número total com `len()`

Tuples em Python

A tupla é semelhante a uma lista, mas com uma diferença fundamental: é imutável. Uma vez criada, não pode ser alterada. Pensa no teu Cartão de Cidadão — os teus dados estão fixos e não podes alterá-los facilmente.

Cofinanciado por:



O que é uma Tupla?

Uma tupla é uma coleção ordenada e imutável. Usa-se quando queremos garantir que os dados não são alterados acidentalmente — como coordenadas geográficas, datas de nascimento, ou configurações fixas.

Características

- Ordenada — tem índices como a lista
- Imutável — não pode ser alterada
- Permite duplicados
- Usa parênteses curvos `()`

Como criar uma tupla

```
# Coordenadas GPS
coordenadas = (38.7169, -9.1399)

# Cores RGB
cor = (255, 128, 0)

# Data de nascimento
nascimento = (15, 3, 2004)

print(coordenadas[0]) # 38.7169
print(type(coordenadas)) # <class 'tuple'>
```

Lista vs Tupla — Qual a diferença?

A diferença principal é a imutabilidade. Vamos ver o que acontece quando tentamos alterar cada uma:

Lista — permite alteração 

```
frutas = ["maçã", "banana"]  
frutas[0] = "manga"  
print(frutas) # ['manga', 'banana']  
# Funciona sem erro!
```

Tupla — não permite alteração 

```
coords = (10, 20)  
coords[0] = 99  
# TypeError: 'tuple' object does  
# not support item assignment
```

 Quando usar cada uma?

Usa List

Quando os dados vão
mudar — carrinho de
compras, lista de tarefas

Usa Tuple

Quando os dados são fixos
— coordenadas,
configurações, datas

Exercício Prático — Tupla de Coordenadas

Vamos praticar com tuplas! Segue as instruções abaixo.

→ Cria uma tupla

Cria uma tupla chamada `localizacao` com as coordenadas da tua cidade (latitude e longitude). Podes pesquisar no Google Maps!

→ Tenta alterar

Tenta alterar o primeiro valor da tupla e observa a mensagem de erro. Que tipo de erro dá? Lê a mensagem com atenção!

→ Acede aos elementos

Imprime a latitude (`localizacao[0]`) e a longitude (`localizacao[1]`) separadamente.

→ Cria uma tupla com os teus dados

Cria `dados_pessoais = ("O teu nome", ano_nascimento, "cidade")` e acede a cada elemento

Conjuntos (Set) em Python

Um set (conjunto) é uma coleção não ordenada e sem duplicados. Pensa numa turma de alunos únicos — cada aluno aparece uma única vez, independentemente de quantas vezes o adicionemos.



O que é um Set?

Um set elimina automaticamente os duplicados e não garante nenhuma ordem específica. É perfeito quando precisamos de valores únicos ou queremos fazer operações matemáticas de conjuntos.

Características

- Não ordenado — sem índices
- Sem duplicados — valores únicos
- Mutável — pode ser alterado
- Usa chavetas {}

Exemplo prático

```
# Duplicados são removidos automaticamente
numeros = {1, 2, 3, 3, 2, 1, 4}
print(numeros)  # {1, 2, 3, 4}
```

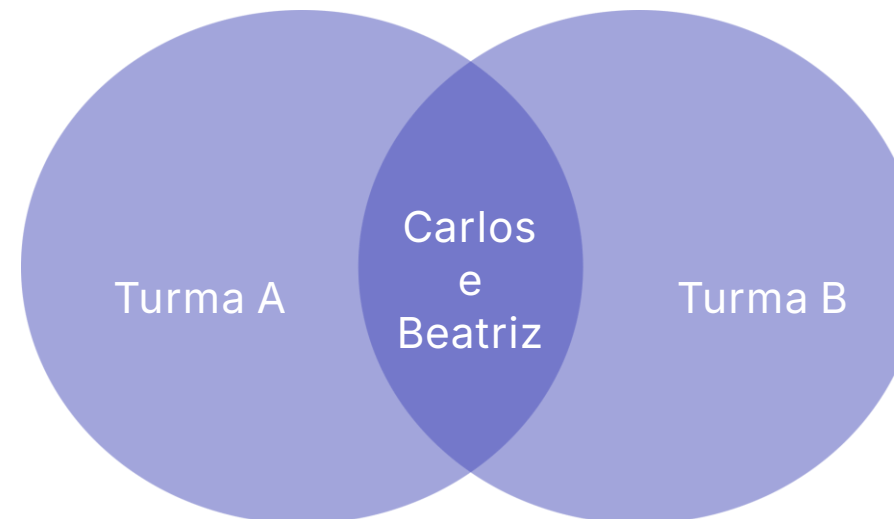
```
# Alunos numa turma
turma = {"Ana", "Rui", "Ana", "Carlos"}
print(turma)  # {'Ana', 'Rui', 'Carlos'}
```

```
# Quantos alunos únicos?
print(len(turma))  # 3
```

Operações com Sets

Os sets permitem fazer operações matemáticas de conjuntos — como união e interseção. São extremamente úteis para comparar grupos de dados.

```
turma_a = {"Ana", "Rui", "Carlos", "Beatriz"}  
turma_b = {"Carlos", "Beatriz", "Diana", "Eduardo"}  
  
# União – todos os alunos (sem repetição)  
todos = turma_a | turma_b  
print("União:", todos)  
  
# Interseção – alunos em ambas as turmas  
comuns = turma_a & turma_b  
print("Interseção:", comuns)  # {'Carlos', 'Beatriz'}  
  
# Diferença – só na turma A  
so_a = turma_a - turma_b  
print("Só turma A:", so_a)  # {'Ana', 'Rui'}
```



Cofinanciado por:



Exercício Prático — Eliminar Duplicados

Um dos usos mais comuns dos sets é remover duplicados de uma lista. Vamos praticar!

O problema

```
# Lista com números repetidos
votos = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
print("Original:", votos)
print("Total votos:", len(votos))

# Converter para set remove duplicados
votos_unicos = set(votos)
print("Únicos:", votos_unicos)
print("Valores únicos:", len(votos_unicos))
```

O teu exercício

1. Cria uma lista `emails` com pelo menos 6 emails, onde 2 ou 3 se repetem
2. Converte para set para obter emails únicos
3. Imprime quantos emails únicos existem

Dicionários em Python

O dicionário é uma das estruturas mais poderosas do Python. Funciona como uma ficha de cliente — cada informação tem um nome (chave) e um valor associado. Por exemplo: *nome* → "Ana", *idade* → 20.

Cofinanciado por:

O que é um Dicionário?

Um dicionário guarda pares de chave → valor. Acedemos aos dados pela chave, não por índice. É como procurar uma palavra num dicionário real — usamos a palavra para encontrar a definição.

Características

- Pares chave → valor
- Acesso pela chave (não por índice)
- Mutável — pode ser alterado
- Chaves únicas e sem duplicados
- Usa chavetas {} com :

Como criar um dicionário

```
# Ficha de aluno
aluno = {
    "nome": "Ana",
    "idade": 20,
    "curso": "Informática"
}

print(aluno["nome"]) # Ana
print(aluno["idade"]) # 20
print(type(aluno)) # <class 'dict'>
```

Manipular Dicionários

Podemos adicionar, modificar e remover pares chave-valor facilmente. Vamos ver as operações mais comuns:

```
aluno = {"nome": "Ana", "idade": 20}

# Adicionar nova chave
aluno["curso"] = "Informática"
print(aluno)  # {'nome': 'Ana', 'idade': 20, 'curso': 'Informática'}

# Modificar valor existente
aluno["idade"] = 21
print(aluno["idade"])  # 21

# Remover uma chave
del aluno["curso"]
print(aluno)  # {'nome': 'Ana', 'idade': 21}

# Ver todas as chaves e valores
print(aluno.keys())    # dict_keys(['nome', 'idade'])
print(aluno.values())  # dict_values(['Ana', 21])
```

Cofinanciado por:



Exercício Prático — Ficha de Utilizador

Vamos criar uma ficha completa de utilizador usando um dicionário. Este exercício simula uma situação real de uma aplicação.

1

Criar a ficha

Cria um dicionário `utilizador` com: `nome`, `email`, `idade` e `cidade`

2

Imprimir dados

Acede e imprime cada valor individualmente usando as chaves respectivas

3

Atualizar

Altera a cidade do utilizador para uma cidade diferente e imprime a ficha atualizada

4

Adicionar campo

Adiciona uma nova chave `"ativo"` com o valor `True` e imprime todas as chaves com `.keys()`

Comparação entre Estruturas

Agora que já conhecemos as quatro estruturas, vamos compará-las lado a lado para entender quando usar cada uma. Esta tabela é a tua referência rápida!

Característica	List	Tuple	Set	Dict
Ordenada?	✓ Sim	✓ Sim	✗ Não	✓ Sim (Python 3.7+)
Mutável?	✓ Sim	✗ Não	✓ Sim	✓ Sim
Duplicados?	✓ Permite	✓ Permite	✗ Não	✗ Chaves únicas
Acesso por	Índice	Índice	Iteração	Chave
Sintaxe	[...]	(...)	{...}	{k: v}

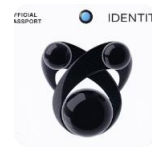
Quando Usar Cada Estrutura? 🤔

A escolha certa da estrutura torna o código mais simples, mais rápido e mais fácil de ler. Usa este guia como referência:



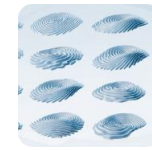
Usa List quando...

Precisas de guardar múltiplos itens em sequência que podem mudar. Ex: lista de tarefas, carrinho de compras, histórico de ações.



Usa Tuple quando...

Os dados são fixos e não devem mudar. Ex: coordenadas GPS, configurações de sistema, dias da semana.



Usa Set quando...

Precisas de valores únicos ou fazer operações de conjuntos. Ex: emails únicos, categorias, comparar dois grupos.



Usa Dict quando...

Os dados têm nomes e valores associados. Ex: ficha de cliente, configuração de app, contagem de palavras.



Hora de Praticar! 💪

Chegou o momento mais importante da sessão. Vamos aplicar tudo o que aprendemos em exercícios práticos que simulam situações reais. Cada exercício usa uma ou mais estruturas.

Cofinanciado por:





Exercício 1 — Lista de Compras Interativa

Vamos criar uma lista de compras completa com as operações mais comuns. Este exercício resume tudo o que aprendemos sobre listas.

```
# Lista de compras inicial
compras = ["leite", "pão", "ovos", "queijo"]
print("Lista inicial:", compras)

# Adicionar produtos novos
compras.append("manteiga")
compras.append("iogurte")
print("Após compras:", compras)

# Remover produto já comprado
compras.remove("pão")
print("Pão removido:", compras)

# Ordenar e contar
compras.sort()
print("Ordenada:", compras)
print("Total:", len(compras), "produtos")
```

Tenta tu!

Adapta o exercício com os teus próprios produtos e:

1. Adiciona pelo menos 3 produtos
2. Remove 1 produto que já "compraste"
3. Acede ao segundo produto por índice
4. Imprime apenas os primeiros 3 usando slicing

Cofinanciado por:





Exercício 2 — Eliminar Duplicados com Set

Um cenário muito comum em programação: recebemos dados com repetições e precisamos de os limpar. O set é a ferramenta perfeita!

```
# Lista de inscrições com repetições
inscricoes = [
    "ana@email.com",
    "rui@email.com",
    "ana@email.com",
    "carlos@email.com",
    "rui@email.com",
    "diana@email.com"
]

print("Total inscrições:", len(inscricoes))

# Remover duplicados
emails_unicos = set(inscricoes)
print("Únicos:", len(emails_unicos))
print(emails_unicos)

# Voltar a lista limpa
lista_final = list(emails_unicos)
print("Lista final:", lista_final)
```

A tua missão

Simula uma votação onde as pessoas podem votar mais do que uma vez:

1. Cria uma lista `votos` com 10 nomes onde alguns se repetem
2. Usa set para descobrir quantos candidatos únicos existem
3. Imprime os candidatos únicos



Exercício 3 — Ficha de Aluno com Dict

Vamos criar uma ficha de aluno completa usando um dicionário — muito parecido com o que acontece em sistemas reais de gestão escolar.

```
# Ficha de aluno
aluno = {
    "nome": "Beatriz Santos",
    "idade": 19,
    "curso": "Programação Web",
    "nota_media": 14.5,
    "ativo": True
}

# Apresentar a ficha
print("=== FICHA DE ALUNO ===")
print("Nome:", aluno["nome"])
print("Idade:", aluno["idade"])
print("Curso:", aluno["curso"])
print("Média:", aluno["nota_media"])
print("Ativo:", aluno["ativo"])

# Atualizar nota
aluno["nota_media"] = 15.2
print("\nNota atualizada:", aluno["nota_media"])
```

Cria a tua ficha!

1. Cria um dicionário `aluno` com os teus próprios dados (podes inventar!)
2. Imprime cada campo com uma etiqueta formatada
3. Atualiza a tua idade
4. Adiciona um campo `"turma"`
5. Imprime todas as chaves com `.keys()`



Exercício 4 — Mini Sistema de Alunos



Este é o exercício mais completo da sessão. Vamos combinar uma lista com dicionários para simular um pequeno sistema de gestão de alunos — é assim que se constrói software real!

```
# Lista de alunos – cada aluno é um dicionário
turma = [
    {"nome": "Ana", "idade": 20, "curso": "Python"},
    {"nome": "Rui", "idade": 22, "curso": "Web"},
    {"nome": "Carlos", "idade": 19, "curso": "Python"},
    {"nome": "Diana", "idade": 21, "curso": "Web"}
]

# Ver todos os alunos
print("=== TURMA ===")
for aluno in turma:
    print(aluno["nome"], "-", aluno["curso"])

# Adicionar novo aluno
novo = {"nome": "Eva", "idade": 20, "curso": "Python"}
turma.append(novo)
print("\nTotal de alunos:", len(turma))
```

📄 💡 **Desafio extra:** Consegues imprimir apenas os alunos do curso "Python"? Usa um `if` dentro do ciclo `for`. Este padrão — lista de dicionários — é usado em praticamente todas as aplicações reais!

Resumo Visual das 4 Estruturas

Antes de terminar, vamos consolidar tudo numa visão geral clara e rápida.

List

Tuple

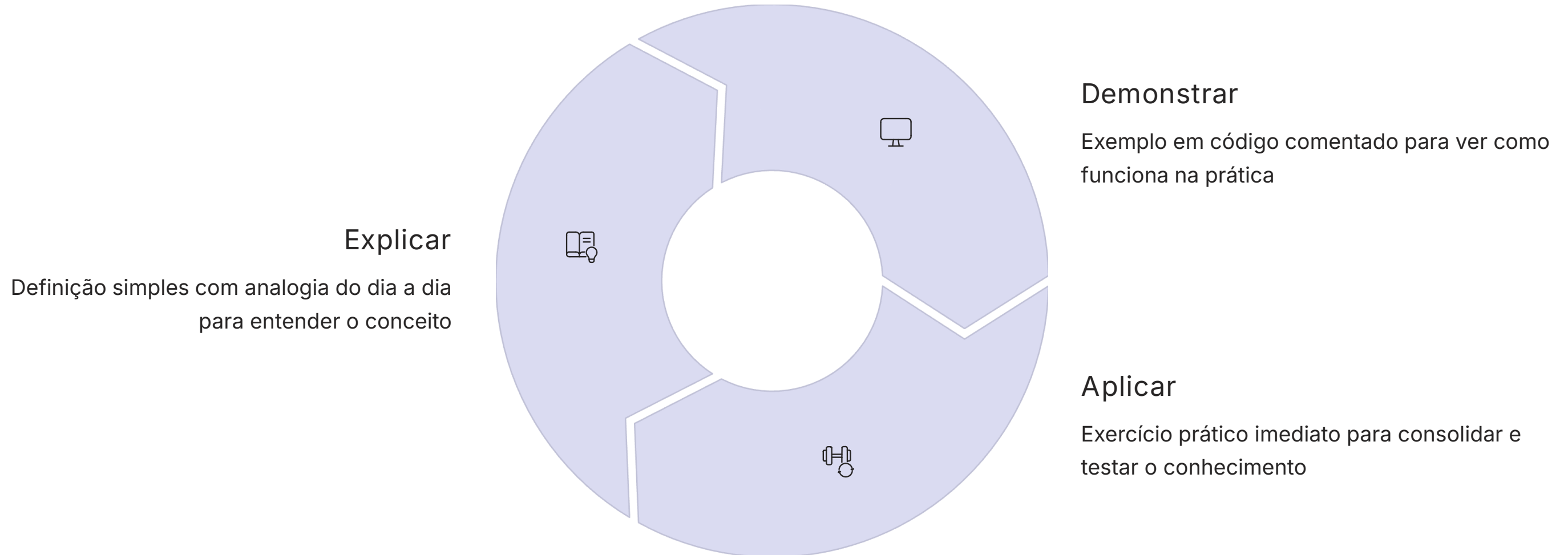
Set

Dict

Cofinanciado por:

O Ciclo de Aprendizagem que Seguimos

Durante toda a sessão, seguimos sempre o mesmo ciclo pedagógico. Reconheces-te nele?



 Este ciclo — Explicar → Demonstrar → Aplicar — é a base da aprendizagem eficaz em programação. Sempre que aprenderes algo novo, repete este ciclo por conta própria!

Erros Comuns a Evitar

Todos os programadores cometem estes erros no início. Conhecê-los com antecedência vai poupar-te muito tempo!

Índices começam em 0, não em 1

`lista[0]` é o primeiro elemento — não `lista[1]`. É o erro mais comum de todos!

Tentar alterar uma Tuple

As tuplas são imutáveis — qualquer tentativa de alteração dá `TypeError`. Usa lista se precisares de modificar.

Aceder a índices num Set

Os sets não têm índices — `meu_set[0]` dá erro. Para aceder a elementos, usa um ciclo `for`.

Chave inexistente num Dict

`aluno["telefone"]` dá `KeyError` se a chave não existir. Usa `aluno.get("telefone")` como alternativa segura.

Dicas de Boas Práticas

Escrever código que funciona é bom — escrever código claro e bem organizado é ainda melhor. Estas dicas vão tornar-te um programador mais profissional desde o início.

Usa nomes descritivos

Em vez de `l = [...]`, escreve `lista_alunos = [...]`. O nome deve dizer o que a variável contém.

Comenta o teu código

Usa `#` para explicar o que cada parte faz. O teu "eu futuro" vai agradecer quando reler o código semanas depois.

Escolhe a estrutura certa desde o início

Antes de criar uma variável, pergunta: "Este dado vai mudar? Tem duplicados? Tem nome e valor?" A resposta guia a escolha.

Testa sempre com `print()`

Após cada operação importante, usa `print()` para verificar se o resultado é o esperado. É o teu melhor amigo no debugging!

O que Aprendeste Hoje?

Fizeste um percurso completo pelas quatro estruturas de dados fundamentais do Python. Verifica se consegues responder a cada ponto:



List

Criar, aceder por índice, fazer slicing e usar `append()`, `remove()`, `sort()`



Tuple

Criar, aceder por índice e perceber porque é imutável e quando usar



Set

Criar, eliminar duplicados, e fazer união e interseção de conjuntos



Dict

Criar, aceder por chave, adicionar, modificar e remover pares chave-valor

Objetivos Alcançados

No início da sessão definimos objetivos claros. Vamos verificar se os atingiste!

4/4

Estruturas

List, Tuple, Set e Dict — todas exploradas com exemplos práticos

4/4

Exercícios

Quatro exercícios práticos realizados, do simples ao mais completo

1/1

Projeto Mini

Sistema de alunos combinando list + dict — um projeto real e funcional

  Se fizeste todos os exercícios, já és capaz de criar e manipular as quatro estruturas fundamentais do Python. Isso é uma base sólida para tudo o que vem a seguir!

Próximos Passos

Com as estruturas de dados dominadas, abre-se um mundo de possibilidades. Eis o que vem a seguir na tua jornada de programação:



Ciclos com Estruturas

Usar `for` para percorrer listas, sets e dicionários automaticamente



Ficheiros e Dados

Ler e escrever dados em ficheiros CSV e JSON usando as estruturas que aprendeste



Funções

Criar funções que recebem e devolvem estruturas de dados — reutilizar código



Projeto Completo

Construir uma aplicação real que usa todas as estruturas em conjunto

Parabéns! Chegaste ao Fim 🎉

Hoje fizeste um salto enorme na tua jornada como programador. Dominar as estruturas de dados é um dos passos mais importantes em Python — e tu fizeste-o!



"Não aprendes a programar lendo sobre programação — aprendes programando." Continua a praticar, experimenta os exercícios em casa, e tenta criar os teus próprios exemplos do quotidiano.



List ✓

Tuple ✓

Set ✓

Dict ✓

